

Module 7 Week A — Integration Task: Domain-Shift Analysis

In Monday's lab you fine-tuned DistilBERT on app reviews and pushed your model to Hugging Face Hub. In this Integration Task, you apply that model to a *different* domain — tech / entertainment news — and analyze where the model generalizes versus where it breaks. The deliverable is a predictions file, a Makefile that reproduces the run, and a one-page memo characterizing the domain-shift behavior.

This is the first piece of the Module 7 evaluation report deliverable. The transfer gap you observe here is the engineering question Week 8 revisits explicitly when comparing fine-tuned vs. pre-trained models on QA and summarization.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

What You Will Produce

Committed to your repo:

- apply.py** — your implementation that loads the model from your Hugging Face Hub URL and applies it to the tech news corpus.
- Makefile** — provided in starter; you verify `make apply` and `make smoke` work locally.
- README.md** — updated with your Hugging Face Hub model URL and the local run command.
- predictions.csv** — 1,033 rows, one per tech-news article, with `article_id`, `text_excerpt`, `predicted_label`, `predicted_probability`.
- domain-shift-analysis.md** — a 1–1.5 page memo (structure given below).

No model file. The model lives at Hugging Face Hub; it's loaded at runtime by `apply.py`.

Setup

- Accept the assignment**

- Clone:

```
git clone <your-repo-url>
cd <your-repo-name>
```

- Install dependencies:

```
pip install -r requirements.txt
```

- Configure your model URL. Copy `.env.example` to `.env` and edit:

```
cp .env.example .env
```

Inside `.env`:

```
MODEL_HUB_ID=<your-hf-username>/m7-app-review-sentiment
```

Use your actual Hugging Face username from Lab 7A. The starter `apply.py` reads `MODEL_HUB_ID` from the environment.

- Create your working branch:

```
git checkout -b integration-7a-domain-shift
```

Note: `.env` is gitignored. You will not commit your environment file. The `MODEL_HUB_ID` value goes in your `README.md` (in plain text) and in your PR description so the TA can find your model.

What You're Building

`apply.py` exposes:

- `load_classifier(model_hub_id)` — load via `AutoModelForSequenceClassification.from_pretrained(...)` and `AutoTokenizer.from_pretrained(...)`.
- `predict(text, model, tokenizer)` — return `(predicted_label, predicted_probability)` for a single text.
- `apply_to_corpus(csv_path, model_hub_id, output_path)` — apply the model to every row of the corpus, write the predictions CSV.
- `main()` — read `MODEL_HUB_ID`, `CORPUS_PATH`, `OUTPUT_PATH` from the environment (with defaults); call `apply_to_corpus`.

The starter has `main()` written and ready; you implement `load_classifier`, `predict`, `apply_to_corpus`.

Task 1: Load the Classifier

```
def load_classifier(model_hub_id: str) -> tuple:
    """
    Load model and tokenizer from a Hugging Face Hub repo id (e.g., 'username/model').

    Returns (model, tokenizer).

    """
    # TODO: AutoModelForSequenceClassification.from_pretrained(model_hub_id)
    # TODO: AutoTokenizer.from_pretrained(model_hub_id)
    # TODO: return both
```

The model and tokenizer download from HF Hub on first call (cached afterwards). The download is automatic — no `huggingface-cli login` required for *reading* a public model.

Task 2: Predict

```
def predict(text: str, model, tokenizer) -> tuple:
    """
    Predict label and probability for a single string.

    Returns (predicted_label_name, predicted_probability).

    """
    # TODO: tokenize text with truncation, max_length=128, return_tensors="pt"
    # TODO: forward pass under torch.no_grad()
    # TODO: softmax the logits along the last dim
    # TODO: get argmax index and the probability at that index
    # TODO: convert the index to a label name using model.config.id2label
    # TODO: return (label_name, float(probability))
```

Read the label name from `model.config.id2label`. Do not hard-code class names — the autograder checks for this. The CI smoke test substitutes a different (binary) public model, and a hard-coded 3-class list would mislabel its outputs silently.

Task 3: Apply to the Corpus

The corpus CSV (`data/tech_news_articles.csv` and the smoke fixture) has four columns: `article_id`, `text`, `category_keyword`, `source`. The model reads the `text` column for each row; the other columns are passed through (you write `article_id` to the output and slice `text` to build `text_excerpt`).

```
def apply_to_corpus(csv_path: str, model_hub_id: str, output_path: str) -> None:
    """
    Read csv_path (columns: article_id, text, category_keyword, source),
    predict for every row using the 'text' column as model input,
    write predictions to output_path.

    Output columns: article_id, text_excerpt, predicted_label, predicted_probability.
    text_excerpt is the first 200 characters of the article text.

    """
    # TODO: load the model and tokenizer once
    # TODO: read the CSV with pandas
    # TODO: iterate over rows, calling predict() on the 'text' column
    # TODO: build a DataFrame with the four output columns
    # TODO: write to output_path with index=False
```

Loading the model is slow (~10–30 seconds the first time, then cached). Loading once and iterating is much faster than re-loading for each row.

Reproducibility — the Makefile

The starter `Makefile` exposes three targets:

```
apply: ## Load model from $MODEL_HUB_ID, apply to data/tech_news_articles.csv,
python apply.py

smoke: ## CI target: apply against substitute public model on a 5-row fixture
MODEL_HUB_ID=distilbert-base-uncased-finetuned-sts-2-english \
CORPUS_PATH=data/tech_news_articles_smoke.csv \
OUTPUT_PATH=predictions_smoke.csv \
python apply.py

clean: ## remove generated predictions
rm -f predictions.csv predictions_smoke.csv
```

You do not modify the Makefile. You verify both targets work:

```
make smoke # runs in ~30 seconds; produces predictions_smoke.csv with 5 rows
make apply # runs in ~3–5 minutes; produces predictions.csv with 1,033 rows (if CI passes)
```

The CI runs `make smoke` automatically. Your committed `predictions.csv` is the real run, against your fine-tuned model.

Writing the Domain-Shift Analysis

A `domain-shift-analysis.md` template is provided in your repo with the section headers and TODO placeholders shown below — fill in each section in place (don't create a new file). The autograder checks for section presence and length; the TA reads your content.

```
# Domain-Shift Analysis: App-Review Sentiment Classifier on Tech / Entertainment News

## Prediction distribution
How many positive / neutral / negative predictions did the model make on the 1,033 tech news articles? A small markdown table with three rows.

## Confidence distribution
Summarize the 'predicted_probability' column from 'predictions.csv' — that's the A small histogram (committed as PNG) is welcome but optional.

## Five qualitative examples
Pick five tech news articles that, together, illustrate domain shift. For each:
- Article ID and a short excerpt
- Predicted label and probability
- Your interpretation: is this prediction reasonable, suspicious, or clearly wrong? What domain-shift pattern does it expose?

The five examples should not be random — pick ones that surface different failure modes (over-confident wrong predictions, hedging on ambiguous text, systematic bias toward one class on a particular topic).

## Engineering judgment
One paragraph: would you ship this model to production for tech / entertainment news classification? Why or why not? Be concrete — calibration, confidence thresholding, the cost of false positives in this specific domain.
```

The “engineering judgment” paragraph is what TAs read most carefully. Don't write a generic “I would not ship because the model wasn't trained on news prose.” Write something a senior engineer would write — specific, grounded in your actual results, addressing real production concerns.

README.md updates

The starter README has placeholders. Add at minimum:

- Your Hugging Face Hub model URL (the same one you put in your Lab 7A evaluation report).
- The exact command to reproduce your `predictions.csv` locally:

```
cp .env.example .env # then edit MODEL_HUB_ID
pip install -r requirements.txt
make apply
```

- A 1–2 paragraph note on what the model was trained on and why you're applying it to tech / entertainment news here.

The autograder checks that your README is meaningfully longer than the starter template and contains a `huggingface.co/` URL.

Submission

- Verify locally:

```
make smoke # should exit 0 and produce predictions_smoke.csv with 5 rows
make apply # should produce predictions.csv with 1,033 rows
pytest tests/ -v
```

- Commit and push the branch.

- Open a PR from `integration-7a-domain-shift` into `main`.

- Wait for the autograder to complete.

Your PR description must include:

- Hugging Face Hub model URL.**
- Prediction distribution** (3 lines: count by class).
- Confidence summary** (1 line: mean predicted probability).
- Engineering judgment** (1 sentence: would you ship?).
- Paste your PR URL into TalentLMS → Module 7 → Integration Task 7A to submit this assignment.

Troubleshooting

- OSError: <model> is not the path to a directory containing a config.json when loading from Hub** — your `MODEL_HUB_ID` is wrong. Verify the URL on huggingface.co; the format is `<username>/<model-name>` with no leading `https://`.
- make smoke works in CI but make apply fails locally with auth errors** — public models on HF Hub don't need auth to read. If you're getting auth errors, your model on HF Hub may be set to private. Visit your model page and confirm it is public; change it if not.
- Predictions come back with the wrong label names (e.g., “POSITIVE” / “NEGATIVE” instead of “positive” / “neutral” / “negative”)** — you ran `make smoke` (which uses a binary substitute model) and saved its output as `predictions.csv`. Run `make apply` (which uses your fine-tuned 3-class model) for the real submission. The autograder checks the row count (1,033) and column names; using the smoke output for submission will fail those tests.
- predictions.csv has 1,033 rows but the analysis memo references only 5 articles** — that's correct; you analyze 5 selected articles in detail, not all 1,033.

Challenge Extensions

The base assignment is complete and gradable on its own.

Challenge Tier 1 — Confidence Calibration Threshold

Add a section to your analysis: at what predicted-probability threshold would you accept the model's predictions as “trusted” vs. “needs human review”? Pick a threshold, justify it with the confidence distribution you observed, and report what fraction of the corpus would land in each bucket.

Challenge Tier 2 — Side-by-Side With a Public Sentiment Model

Pick a public sentiment classifier that on Hugging Face Hub (e.g., `cardiffnlp/twitter-roberta-base-sentiment-latest` — note that it has different label names; you'll need to handle the mapping). Apply it to the same 1,033 tech news articles. Compare predictions article-by-article: how often do they agree? When they disagree, which one looks more reasonable? Add a `model-comparison.md` memo.

Challenge Tier 3 — Build an Inspection UI

Build a tiny script (`inspect.py`) that takes an article ID, runs the model, and outputs the prediction, the top-3 class probabilities, and a per-token attention visualization (using `transformers` attention output and a simple `matplotlib` heatmap or HTML render). Useful for the Tuesday Honors stretch's calibration analysis. The visualization piece is the engineering work.

